Semana 12: IA generativa y agentes — LLMs, prompting, RAG y agentes

Nivel: Avanzado | **Prerrequisitos:** Semana 11 (redes neuronales, ciclo de entrenamiento, PyTorch), Semana 10 (entrenamiento/prueba, sobreajuste), Semana 8 (probabilidad, semilla), Semana 7 (producto punto, similitud coseno), Semana 6 (`pytest`, Git, funciones como objetos) y Semana 4 (diccionarios y listas) | **Versión:** v1.0 | **Fecha:** 2026-11-09

Objetivos de aprendizaje

1. Explicar qué es un **LLM** (modelo que predice el siguiente token) y cómo se relaciona con las redes de la Semana 11.
2. Escribir **prompts** efectivos (rol, contexto, tarea, formato) y reconocer sus límites, incluidas las **alucinaciones**.
3. Describir el flujo de **RAG** (recuperar → aumentar → generar) e implementar una versión mínima con similitud coseno.
4. Explicar qué es un **agente** (un LLM en un bucle que decide y usa herramientas) e implementar uno con límite de pasos.
5. Probar con `pytest` código que depende de un LLM, sustituyendo el modelo por un doble de prueba.

1. Qué es un LLM

En la Semana 11 una red aprendía XOR ajustando pesos. Un **LLM** (Large Language Model, "modelo de lenguaje grande") es la misma idea escalada: una red profunda con miles de millones de pesos entrenada con una tarea simple, **predecir el siguiente token**.

Analogía: el autocompletado del teléfono, pero que leyó una biblioteca entera. El teclado sugiere la siguiente palabra; un LLM hace lo mismo con tanto contexto y tan buen entrenamiento que puede redactar, resumir, traducir o escribir código.

- **Token:** trozo de texto (una palabra corta o parte de una larga). El modelo lee y escribe tokens, no letras.
- **Ventana de contexto:** cuántos tokens cabe leer a la vez; lo que no cabe, el modelo no lo ve.
- **Salida probabilística:** el modelo asigna probabilidades (Semana 8) a los tokens posibles y elige uno. La **temperatura** controla cuánto se arriesga: baja = predecible, alta = variada. Por eso la misma pregunta puede dar respuestas distintas.

- **Preentrenamiento y ajuste:** primero aprende lenguaje con texto masivo; luego se refina para seguir instrucciones. Tú casi nunca entrenas un LLM: lo **usas** por una API o lo ejecutas ya entrenado.

> **Jerga traducida:** "Transformer" = la arquitectura de red que usan los LLM, que pondera qué partes del texto importan para cada palabra (atención). "Inferencia" = usar el modelo ya entrenado para responder.

Límite esencial: un LLM produce texto **plausible**, no necesariamente **verdadero**. Cuando inventa datos con aplomo se llama **alucinación**.

2. Prompting: pedir bien

El **prompt** es la instrucción que le das. Un prompt claro tiene cuatro partes:

Parte	Pregunta que responde	Ejemplo
Rol	¿Quién eres?	"Eres un tutor de Python para principiantes."
Contexto	¿Qué debes saber?	"El alumno ya vio listas y bucles."
Tarea	¿Qué hago?	"Explica qué hace `enumerate`."
Formato	¿Cómo respondo?	"Máximo 3 líneas y un ejemplo."

Técnicas útiles: dar **ejemplos** dentro del prompt (few-shot), pedir pasos de razonamiento en problemas difíciles, y pedir salida estructurada (JSON) cuando otro programa la va a leer. Tratar el prompt como código: se versiona con Git (Semana 6) y se prueba.

3. Usar un LLM desde Python

```
# llamada_llm.py — requiere `pip install anthropic` y la variable de entorno
ANTHROPIC_API_KEY
import anthropic

cliente = anthropic.Anthropic()          # lee la clave desde el entorno, nunca del
código
respuesta = cliente.messages.create(
    model="claude-sonnet-5-5",           # revisa la documentación para el modelo
vigente
    max_tokens=300,
    system="Eres un tutor de Python para principiantes.",
    messages=[{"role": "user", "content": "Explica qué hace enumerate en 3 líneas."}],
```

```
)
print(respuesta.content[0].text)
```

Este fragmento no se ejecutó en la verificación de la clase (requiere clave y red); los ejemplos de abajo sí se ejecutaron. Regla de seguridad: **nunca escribas la clave en el código ni la subas a Git**; usa variables de entorno.

4. RAG: darle al modelo tus propios datos

Un LLM no conoce tus documentos y puede alucinar. **RAG** (Retrieval-Augmented Generation, "generación aumentada con recuperación") resuelve esto en tres pasos:

1. **Recuperar:** buscar los fragmentos de tus documentos más parecidos a la pregunta.
2. **Aumentar:** pegarlos en el prompt como contexto.
3. **Generar:** el modelo responde apoyándose en ese contexto.

Analogía: examen a libro abierto. El modelo no memoriza el libro; busca la página correcta y responde con ella delante.

¿Cómo se mide "parecido"? Con **embeddings**: vectores que representan el significado de un texto, de modo que textos similares quedan cerca. La cercanía se mide con la **similitud coseno** de la Semana 7. Los sistemas reales usan un modelo de embeddings y una base de datos vectorial; la lógica es idéntica a la del ejemplo práctico (que usa una bolsa de palabras de juguete para correr sin dependencias).

5. Agentes: un LLM que actúa

Un **agente** es un LLM dentro de un bucle: observa, decide, **usa una herramienta** (calcular, buscar, leer un archivo, llamar una API), ve el resultado y repite hasta responder.

Analogía: un asistente con un teléfono y una calculadora. No sabe todo, pero sabe cuándo usar cada herramienta.

```
repetir (máximo N pasos):
    el modelo decide: ¿responder o usar una herramienta?
    si responde → terminar
    si pide una herramienta → ejecutarla (tu código) y devolverle el resultado
```

Puntos de diseño que importan:

- **El modelo propone, tu código ejecuta.** Valida el nombre y los argumentos de cada herramienta.
- **Lista cerrada de herramientas** y permisos mínimos: no des acceso de borrado "por si acaso".

- **Tope de pasos** para evitar bucles infinitos y gastos descontrolados.
- **Confirmación humana** antes de acciones irreversibles (enviar correos, pagar, borrar).
- **Inyección de prompt:** un texto externo (una página web, un correo) puede contener instrucciones maliciosas. Trata todo contenido recuperado como **datos, no como órdenes**.

6. Por qué esto importa para IA/ML

- **Es la capa de producto del Deep Learning:** los LLM son redes de la Semana 11 a gran escala; aplicarlos bien es hoy una habilidad central.
- **RAG une todo el curso:** vectores y coseno (Semana 7), diccionarios (Semana 4), búsqueda y complejidad (Semana 5) y evaluación (Semana 10).
- **Evaluar no es opcional:** un sistema de IA generativa necesita pruebas, casos de referencia y métricas, igual que un modelo clásico.
- **Siguiente paso profesional:** orquestación de agentes, evaluación de LLM, seguridad y costos; aquí tienes los cimientos.

7. Ejemplo práctico: RAG mínimo y agente con herramientas

Pseudocódigo (RAG)

```
convertir cada documento y la pregunta en vectores
calcular la similitud coseno pregunta vs. cada documento
ordenar de mayor a menor y quedarse con los k mejores (si similitud > 0)
armar el prompt: instrucciones + contexto recuperado + pregunta
```

Python: RAG

```
# rag_minimo.py
"""RAG mínimo: recuperar los fragmentos más parecidos a la pregunta y armar el
prompt."""
import math
import re
from collections import Counter

DOCUMENTOS = [
    "Una época es una pasada completa por todos los datos de entrenamiento.",
    "El sobreajuste ocurre cuando el modelo memoriza el entrenamiento y falla con datos
nuevos.",
```

```
    "La tasa de aprendizaje controla qué tan grande es cada paso del descenso del
gradiente.",
    "Git guarda versiones del código mediante commits.",
]


def tokenizar(texto):
    return re.findall(r"\w+", texto.lower())


def vectorizar(texto):
    # "Embedding" de juguete: cuántas veces aparece cada palabra (bolsa de palabras).
    return Counter(tokenizar(texto))


def similitud_coseno(a, b):
    # Mismo producto punto de la Semana 7, sobre diccionarios dispersos.
    punto = sum(a[p] * b[p] for p in a if p in b)
    norma = math.sqrt(sum(v * v for v in a.values())) * math.sqrt(sum(v * v for v in
b.values()))
    return punto / norma if norma else 0.0


def recuperar(pregunta, documentos, k=2):
    q = vectorizar(pregunta)
    puntuados = [(similitud_coseno(q, vectorizar(d)), d) for d in documentos]
    puntuados.sort(key=lambda par: par[0], reverse=True)
    return [d for s, d in puntuados[:k] if s > 0]


def construir_prompt(pregunta, fragmentos):
    contexto = "\n".join(f"- {f}" for f in fragmentos) or "(sin contexto)"
    return (
        "Responde SOLO con el contexto. Si no alcanza, di que no lo sabes.\n"
        f"Contexto:\n{contexto}\n\nPregunta: {pregunta}"
    )


if __name__ == "__main__":
    pregunta = "¿Qué es el sobreajuste del modelo?"
    fragmentos = recuperar(pregunta, DOCUMENTOS, k=1)
    print(fragmentos)
    print(construir_prompt(pregunta, fragmentos))
```

Resultado verificado por ejecución: recupera el fragmento sobre el sobreajuste y construye el prompt con ese contexto.

Pseudocódigo (agente)

```
mensajes = [pregunta]
repetir hasta max_pasos:
    decisión = llm(mensajes)
    si es respuesta → devolverla
    si pide herramienta conocida → ejecutarla y agregar el resultado a mensajes
    si no → informar error
si se agotan los pasos → avisar
```

Python: agente

```
# agente.py
"""Un agente mínimo: el modelo decide, el código ejecuta herramientas y devuelve el
resultado."""
import json


def sumar(a: float, b: float) -> float:
    return a + b


HERRAMIENTAS = {"sumar": sumar}   # lista cerrada: el agente solo puede llamar lo que
registres


def ejecutar_agente(llm, pregunta, max_pasos=5):
    """`llm` es cualquier función mensajes -> dict. Así se prueba sin red ni claves."""
    mensajes = [{"role": "user", "content": pregunta}]
    for _ in range(max_pasos):                      # tope de pasos: evita bucles
infinitos
        decision = llm(mensajes)
        if decision["tipo"] == "respuesta":
            return decision["texto"]
        nombre, args = decision["herramienta"], decision["args"]
        if nombre not in HERRAMIENTAS:
            resultado = f"error: herramienta desconocida '{nombre}'"
        else:
            resultado = HERRAMIENTAS[nombre](**args)
        mensajes.append({"role": "tool", "content": json.dumps({"resultado":
```

```
resultado})})
    return "No pude terminar dentro del límite de pasos."


def llm_simulado(mensajes):
    # Sustituye al modelo real: primero pide la herramienta, luego responde con su
resultado.
    if mensajes[-1]["role"] == "user":
        return {"tipo": "herramienta", "herramienta": "sumar", "args": {"a": 2, "b":
3}}
    resultado = json.loads(mensajes[-1]["content"])["resultado"]
    return {"tipo": "respuesta", "texto": f"La suma es {resultado}."}


if __name__ == "__main__":
    print(ejecutar_agente(llm_simulado, "¿Cuánto es 2 + 3?"))
```

Resultado verificado: `La suma es 5.` El LLM real se inyecta como parámetro (`llm`); aquí se usa uno simulado para probar la lógica sin red ni costos.

Pruebas con pytest (Semana 6)

```
import agente
from rag_minimo import DOCUMENTOS, recuperar, construir_prompt, similitud_coseno,
vectorizar


def test_recupera_el_fragmento_correcto():
    assert "sobreajuste" in recuperar("¿Qué es el sobreajuste?", DOCUMENTOS, k=1)[0]


def test_sin_coincidencias_no_hay_contexto():
    assert recuperar("zzz qqq", DOCUMENTOS) == []


def test_similitud_identica_es_uno():
    v = vectorizar("hola mundo")
    assert abs(similitud_coseno(v, v) - 1.0) < 1e-9


def test_prompt_incluye_contexto_y_pregunta():
    p = construir_prompt("¿X?", ["dato A"])
    assert "dato A" in p and "¿X?" in p
```

```
def test_agente_usa_la_herramienta():
    assert agente.ejecutar_agente(agente.llm_simulado, "2+3") == "La suma es 5."


def test_agente_se_detiene_por_limite_de_pasos():
    bucle = lambda m: {"tipo": "herramienta", "herramienta": "sumar", "args": {"a": 1,
"b": 1}}
    assert "límite" in agente.ejecutar_agente(bucle, "x", max_pasos=3)


def test_herramienta_desconocida_no_se_ejecuta():
    pasos = iter([{"tipo": "herramienta", "herramienta": "borrar_todo", "args": {}},
                  {"tipo": "respuesta", "texto": "ok"}])
    assert agente.ejecutar_agente(lambda m: next(pasos), "x") == "ok"
```

Las 7 pruebas pasan (verificado con Python 3.13 y pytest 9). Observa la técnica: **inyectar el LLM como dependencia** permite probar el bucle, el límite de pasos y el rechazo de herramientas desconocidas sin llamar a ninguna API.

8. Notas de vigencia técnica (2026)

- **Los modelos cambian rápido:** no fijes nombres de modelo en el código; guárdalos en configuración y consulta la documentación del proveedor. Los identificadores de esta clase pueden quedar obsoletos.
- **Contextos largos no eliminan RAG:** meter todo en el prompt es caro y menos preciso; RAG sigue siendo la forma de usar datos propios, privados o cambiantes.
- **Embeddings reales:** usa un modelo de embeddings y una base vectorial (p. ej. pgvector, FAISS, Chroma); la bolsa de palabras de hoy es solo didáctica.
- **Llamada a herramientas y salida estructurada** son ya funciones nativas de las APIs; úsalas en vez de interpretar texto libre.
- **Estándares de herramientas:** el protocolo **MCP** (Model Context Protocol) se usa para conectar agentes con herramientas y datos de forma estándar.
- **Frameworks de agentes:** existen varios (LangGraph, SDKs de agentes de los proveedores); evoluciona rápido, así que entiende el bucle de la sección 5 antes de adoptar uno.
- **Modelos abiertos:** puedes ejecutar modelos abiertos localmente (Ollama, Hugging Face) cuando importen la privacidad o el costo.
- **Seguridad:** claves en variables de entorno, límites de gasto y registro (logging) de cada llamada.

9. Errores comunes de principiante

- **Confiar ciegamente en la respuesta:** verifica hechos, cifras y código generado.
- **Subir claves de API a Git.**
- **Prompts vagos** ("explícame esto") sin rol, contexto ni formato.
- **RAG sin evaluar:** si la recuperación trae fragmentos irrelevantes, el modelo responderá mal aunque sea excelente.
- **Agente sin tope de pasos ni validación** de herramientas.
- **Tratar el texto recuperado como instrucciones** (inyección de prompt).
- **Pegar datos personales o confidenciales** en servicios externos sin revisar la política de privacidad.
- **Usar un LLM para lo que resuelve una función simple** (una suma no necesita un modelo).

10. Ejercicio propuesto

Amplía `agente.py` con una segunda herramienta `multiplicar` y escribe un `llm_simulado` que encadene dos herramientas antes de responder (por ejemplo, `(2 + 3) * 4`). Añade pruebas con `pytest` para el caso normal y para argumentos inválidos (captura el error y devuélvelo al modelo como mensaje en lugar de romper el programa). Versiona con Git.

Pista: ¿qué excepción lanza `HERRAMIENTAS[nombre](**args)` si faltan argumentos?

11. Autoevaluación

1. ¿Qué tarea simple se entrena en un LLM? — Predecir el siguiente token.
2. ¿Qué es una alucinación? — Texto plausible pero falso que el modelo presenta con seguridad.
3. ¿Qué cuatro partes tiene un buen prompt? — Rol, contexto, tarea y formato.
4. ¿Qué problema resuelve RAG? — Que el modelo responda con tus datos actuales o privados sin reentrenarlo, reduciendo alucinaciones.
5. Ordena los pasos de RAG. — Recuperar → aumentar el prompt → generar.
6. ¿Qué mide la similitud coseno entre embeddings? — Qué tan parecido es el significado (cercanía de los vectores).
7. ¿Qué diferencia a un agente de una sola llamada a un LLM? — El agente repite un bucle de decidir, usar herramientas y observar resultados.
8. Menciona dos medidas de seguridad para un agente. — Tope de pasos, lista cerrada de herramientas, validación de argumentos, confirmación humana.
9. V/F: "Si el texto recuperado dice 'ignora tus instrucciones', el agente debe obedecer." — Falso: es inyección de prompt; el contenido externo son datos, no órdenes.

12. Verificación de dependencias

Depende de la Semana 11 (redes neuronales y el ciclo de entrenamiento, base conceptual de los LLM), la Semana 10 (evaluación y sobreajuste), la Semana 8 (probabilidad y semilla), la Semana 7 (producto punto y similitud coseno, usados en la recuperación), la Semana 6 (`pytest`, Git, objetos y funciones), la Semana 5 (ordenar resultados) y la Semana 4 (diccionarios, listas, `Counter`). No requiere nada posterior: es el último módulo del roadmap. La llamada a la API del modelo es un fragmento ilustrativo, no necesario para los ejemplos verificados.